

# Towards C++23 executors: A proposal for an initial set of algorithms

Document #: P1897R3  
Date: 2020-05-16  
Project: Programming Language C++ SG1  
Reply-to: Lee Howes  
<[lwh@fb.com](mailto:lwh@fb.com)>

## § 1 Changelog

### § 1.1 Differences between R2 and R3

- Rename `just_via` to `just_on`.
- Rename `via` to `on`.
- Add `ensure_started`.
- Add note on the feedback about `indexed_for` in Prague. Removed `indexed_for` from the paper of initial algorithms.
- Add `let_value`.
- Tweaked `handle_error` wording to be more similar to `let_value`, and renamed `let_error` for naming consistency.
- Updated to use P0443R13 as a baseline.
- Improves the wording to be closer to mergable wording and less pseudowording.
- Modified `sync_wait` to terminate on done rather than throwing.
- Added notes to clarify execution context of completion of `on` and `just_on`.

### § 1.2 Differences between R1 and R2

- Add `just_via` algorithm to allow type customization at the head of a work chain.
- Add `when_all` to fill missing gap in the ability to join sender chains.
- Add `indexed_for` based on feedback during the Belfast meeting to have a side-effecting algorithm.
- Propose question on replacing `bulk_execute` with `indexed_for` for the Prague meeting.

## § 1.3 Differences between R0 and R1

- Improve examples to be clearer, and fully expanded into function call form.
- Add reference to `range.adapter`.
- Remove `is_noexcept_sender`.
- Remove `just_error`.
- Clarified use of parameter packs of values and errors.
- Removed confusing use of `on` in addition to `via` in the final example.

## § 2 Introduction

In [P0443R13] we have included the fundamental principles described in [P1660R0], and the fundamental requirement to customize algorithms. In recent discussions we have converged to an understanding of the `submit` operation on a sender and its more fundamental primitives `connect` and `start` supporting general interoperability between algorithms, and algorithm customization giving us full flexibility to optimize, to offload and to avoid synchronization in chains of mutually compatible algorithm customizations.

As a starting point, in [P0443R13] we only include a `bulk_execute` algorithm, that satisfies the core requirement to provide scalar and bulk execution. To make the C++23 solution completely practical, we should extend the set of algorithms, however. This paper suggests an expanded initial set that enables early useful work chains. This set is intended to act as a discussion focus for us to discuss one by one, and to analyze the finer constraints of the wording to make sure we do not over-constrain the design.

This paper does not attempt to propose the mapping of the C++20 parallel algorithms into an asynchronous environment. Once we have basic primitives, we can describe default implementations of the parallel algorithms, as well as `std::async`, in terms of these.

In the long run we expect to have a much wider set of algorithms, potentially covering the full set in the current C++20 parallel algorithms. The precise customization of these algorithms is open to discussion: they may be individually customized and individually defaulted, or they may be optionally individually customized but defaulted in a tree such that customizing one is known to accelerate dependencies. It is open to discussion how we achieve this and that is an independent topic, beyond the scope of this paper.

### § 2.1 Summary

Starting with [P0443R13] as a baseline we have the following customization points:

- `connect(sender, receiver) -> operation_state`
- `start(operation_state) -> void`
- `submit(sender, receiver) -> void`

- `schedule(scheduler) -> sender`
- `execute(executor, invocable) -> void`
- `set_done`
- `set_error`
- `set_value`

and the following Concepts:

- `scheduler`
- `receiver`
- `receiver_of`
- `sender`
- `sender_to`
- `typed_sender`
- `operation_state`
- `executor`
- `executor_of`

We propose immediately discussing the addition of the following algorithms:

- `just(v...)`
  - returns a sender of the value(s) `v...`
- `just_on(sch, v...)`
  - a variant of the above that embeds the `on` algorithm
- `on(s, sch)`
  - returns a sender that propagates the value or error from `s` on `sch`'s execution context
- `sync_wait(s)`
  - blocks and returns a `T` of the value type of the sender, throwing on error, and terminates on done.
- `when_all(s...)`
  - returns a sender that completes when all Senders `s...` complete, propagating all values
- `transform(s, f)`
  - returns a sender that applies `f` to the value passed by `s`, or propagates errors or cancellation
- `let_value(s, f)`

- Creates an async scope where the value propagated by `s` is available for the duration of another async operation produced by `f`. Error and cancellation signals propagate unmodified.
- `let_error(s, f)`
  - Creates an async scope where an error propagated by `s` is available for the duration of another async operation produced by `f`. Value and cancellation propagate unmodified.
- `ensure_started(s)`
  - Eagerly submits `s` and returns a sender that may be executing concurrently with surrounding code.

## § 2.2 Examples

### § 2.2.0.1 Simple example

A very simple example of applying a function to a propagated value and waiting for it.

```
auto just_sender = // sender_to<int>
    just(3);

auto transform_sender = // sender_to<float>
    transform(
        std::move(just_sender),
        [](int a){return a+0.5f;});

int result = // value: 3.5
    sync_wait(std::move(transform_sender));
```

In this very simple example we:

- start a chain with the value three
- apply a function to the incoming value, adding 0.5 and returning a sender of a float.
- block for the resulting value and assign the float value 3.5 to `result`.

Using operator `|` as in ranges to remove the need to pass arguments around, we can represent this as:

```
float f = sync_wait(
    just(3) | transform([](int a){return a+0.5f;}));
```

### § 2.2.0.2 Using `when_all`

`when_all` joins a list of incoming senders, propagating their values.

```
auto just_sender = // sender_to<int>
    just(std::vector<int>{3, 4, 5}, 10);
auto just_float_sender = // sender_to<int>
    just(20.0f);
```

```
auto when_all_sender = when_all(
    std::move(just_sender), std::move(just_float_sender));

auto transform_sender(
    std::move(when_all_sender),
    [](std::vector<int> vec, int /*i*/, float /*f*/) {
        return vec;
    })

vector<int> result =          // value: {3, 4, 5}
    sync_wait(std::move(transform_sender));
```

This demonstrates simple joining of senders:

- start a chain with a pack of a vector and an int
- start a second chain with a float
- join the two to produce a pack of a vector, an int and a float
- applies a transform to filter out the vector result
- block for the resulting value and assign vector {3, 4, 5} to result

Using operator `|` as in ranges to remove the need to pass arguments around, we can represent this as:

```
vector<int> result_vec = sync_wait(
    when_all(just(std::vector<int>{3, 4, 5}, 10), just(20.0f)) |
    transform([](vector<int> vec, int /*i*/, float /*f*/) {return
vec;}));
```

### § 2.2.0.3 With exception

A simple example showing how an exception that leaks out of a transform may propagate and be thrown from `sync_wait`.

```
int result = 0;
try {
    auto just_sender = just(3);
    auto on_sender = on(std::move(just_sender), scheduler1);
    auto transform_sender = transform(
        std::move(on_sender),
        [](int a){throw 2;});
    auto skipped_transform_sender = transform(
        std::move(transform_sender).
        []() {return 3;});

    result = sync_wait(std::move(skipped_transform_sender));
} catch(int a) {
    result = a; // Assign 2 to
result
}
```

In this example we:

- start a chain with an int value 3
- switch the context to one owned by scheduler1
- apply a transformation to the value 3, but this transform throws an exception rather than returning a transformed value
- skip the final transform because there is an error propagating
- block for the resulting value, seeing an exception thrown instead of a value returned
- handle the exception

As before, using operator| as in ranges to remove the need to pass arguments around, we can represent this more cleanly:

```
int result = 0;
try {
    result = sync_wait(
        just(3) |
        on(scheduler1) |
        transform([](int a){throw 2;}) |
        transform([](){return 3;}));
} catch(int a) {
    result = a;
}
// Assign 2 to result
```

#### § 2.2.0.4 Handle an exception

Very similar to the above, we can handle an error mid-stream

```
auto just_sender = just(3);
auto via_sender = on(std::move(just_sender), scheduler1);
auto transform_sender = transform(
    std::move(via_sender),
    [](int a){throw 2;});
auto skipped_transform_sender = transform(
    std::move(transform_sender),
    [](){return 3;});
auto error_handling_sender = let_error(
    std::move(skipped_transform_sender),
    [](exception_ptr e){return just(5);});

auto result = sync_wait(std::move(error_handling_sender));
```

In this example we:

- start a chain with an int value 3
- switch the context to one owned by scheduler1
- apply a transformation to the value 3, but this transform throws an exception rather than returning a transformed value

- skip the final transform because there is an error propagating
- handle the error channel, applying an operation to an `exception_ptr` pointing to the value 2
- in handling the error we return a sender that propagates the value 5, thus recovering from the error
- block for the resulting value, assigning 5 to `result`

As before, using `operator|` as in ranges to remove the need to pass arguments around, we can represent this more cleanly:

```
auto s = ;
int result = sync_wait(
    just(3) |
    on(scheduler1) |
    transform([](float a){throw 2;}) |
    transform([](){return 3;}) |
    let_error([](auto e){
        return just(5;)}));
```

## § 3 Impact on the standard library

### § 3.1 Sender adapter objects

Taking inspiration from [range adaptors](#) define sender adapters.

Wording to be based on `[range.adaptors]` with the basic requirement that:

- `operator|` be overloaded for the purpose of creating pipelines over senders
- That the following are equivalent expressions:
  - `algorithm(sender, args...)`
  - `algorithm(args...)(sender)`
  - `sender | algorithm(args...)`
- that `algorithms(args...)` is a *sender adaptor closure object*
- TBD where sender adapters are declared

Details below are in loosely approximated wording and should be made consistent with [\[P0443R11\]](#) and the standard itself when finalized. We choose this set of algorithms as a basic set to allow a range of realistic, though still limited, compositions to be written against executors.

## § 3.2 `execution::just`

### § 3.2.1 Overview

just creates a sender that propagates a value inline to a submitted receiver.

Signature:

```
template <typename T>
concept moveable-value = // exposition only
    move_constructible<remove_cvref_t<T>> &&
    constructible_from<remove_cvref_t<T>, T>;

template <moveable-value... Ts>
see-below just(Ts&&... ts) noexcept(see-below);
```

[ Example:

```
int r = sync_wait(just(3));
// r==3
```

- end example]

## § 3.2.2 Wording

The expression `execution::just(t...)` returns a sender, `s` wrapping the values `t...`.

- When `execution::connect(s, r)` is called resulting in operation\_state `o` containing `rCopy` with type `remove_cvref_t<decltype(r)>` and initialized with `r` and followed by `execution::start(o)` for some `r`, will call `execution::set_value(r, std::move(t)...)`, inline with the caller.
- If moving of `t` throws, then will catch the exception and call `execution::set_error(r, e)` with the caught exception\_ptr.

## § 3.3 execution::just\_on

### § 3.3.1 Overview

`just_on` creates a sender that propagates a value to a submitted receiver on the execution context of a passed scheduler. Semantically equivalent to `on(just(t), s)` if `just_on` is not customized on `s`. Providing `just_on` offers an opportunity to directly customise the algorithm to control allocation of the value `t` at the head of a custom pipeline.

Signature:

```
template <execution::scheduler Sch, moveable-value... Ts>
see-below just_on(Sch sch, Ts&&... ts) noexcept(see-below);
```

[ Example:

```
MyScheduler s;
int r = sync_wait(just_on(s, 3));
// r==3
```

- end example]



### § 3.3.2 Wording

The name `execution::just_on` denotes a customization point object. For some subexpressions `sch` and `ts...` let `Sch` be a type such that `decltype((sch))` is `Sch` and let `Ts...` be a pack of types such that `decltype((ts))...` is `Ts...`. The expression `execution::just_on(sch, ts...)` is expression-equivalent to:

- `sch.just_on(ts...)` if that expression is valid and if `sch` satisfies scheduler.
- Otherwise, `just_on(sch, ts...)`, if that expression is valid, if `sch` satisfies scheduler with overload resolution performed in a context that includes the declaration

```
void just_on() = delete;
```

and that does not include a declaration of `execution::just_on`. \* Otherwise returns the result of the expression: `execution::on(execution::just(ts...), sch)`

- Any customisation of `execution::just_on(sch, ts)` returning a sender `s` shall execute calls to `set_value`, `set_done` or `set_error` on a receiver connected to `s` on an execution context owned by `sch`.

## § 3.4 `execution::sync_wait`

### § 3.4.1 Overview

Blocks the calling thread to wait for the passed sender to complete. Returns `T` when passed a `typed_sender` that sends a `T` on the value channel, where `T` may be `void`, throws if an exception is propagated and calls `std::terminate` on propagation of the `set_done()` signal.

```
template <execution::typed_sender S>
auto sync_wait(S&& s) -> std::sender_traits<S>::value_types;
template <class ValueType, execution::sender S>
ValueType sync_wait_r(S&& s);
```

[ Example:

```
int r = sync_wait(just(3));
float r = sync_wait<float>(just(3.5f));
// r==3
```

- end example]

### § 3.4.2 Wording

The name `execution::sync_wait` denotes a customization point object. For some subexpression `s` let `S` be a type such that `decltype((s))` is `S`. The expression `execution::sync_wait(s)` is expression-equivalent to:

- `s.sync_wait()` if that expression is valid and if `S` satisfies sender.

- Otherwise, `sync_wait(s)`, if that expression is valid, if `S` satisfies sender, with overload resolution performed in a context that includes the declaration

```
void sync_wait() = delete;
```

and that does not include a declaration of `execution::sync_wait`.

- Otherwise constructs a receiver, `r` over an implementation-defined synchronization primitive and passes `r` to `execution::connect(s, r)` returning some operation\_state `os`. Waits on the synchronization primitive to block on completion of `s`.
  - If the operation completes by calling `set_value(r, t)` then `sync_wait()` will return a value, `x`, of type `remove_cvref_t<decltype(t)>`.
  - If the operation completes by calling `set_value(r)` then `sync_wait()` will return `void`.
  - If the operation completes by calling `set_error(r, e)` then `sync_wait()` calls `std::rethrow_exception(e)` if `decltype(e)` is `std::exception_ptr` or `throw e`; otherwise.
  - If the operation completes by calling `set_done(r)` then `sync_wait()` will call `std::terminate`.

## § 3.5 execution::on

### § 3.5.1 Overview

Takes a sender and a scheduler and ensures that the sender operation is connected and started on the execution context associated with the scheduler, giving the programmer control over where the work encapsulated by sender is started.

```
template <execution::sender S, execution::scheduler Sch>
see-below on(S s, Sch sch);
```

[ Example:

```
auto r = sync_wait(just(3) | on(my_scheduler{}) | transform([](int
v){return v+1;});
// r==3
```

- end example]

### § 3.5.2 Wording

The name `execution::on` denotes a customization point object. For some subexpressions `s` and `sch`, let `S` be a type such that `decltype((s))` is `S` and `Sch` be a type such that `decltype((sch))` is `Sch`. The expression `execution::on(s, sch)` is expression-equivalent to:

- `s.on(sch)` if that expression is valid, if `S` satisfies sender

- Otherwise, `on(s, sch)` if that expression is valid, and if `S` satisfies sender and if `Sch` satisfies scheduler, with overload resolution performed in a context that includes the declaration

```
void on() = delete;
```

and that does not include a declaration of `execution::on`.

- Otherwise:
  - Constructs a sender `s2` such that when `connect` is called with some receiver `output_receiver` as `execution::connect(s2, output_receiver)` resulting in an `operation_state` `os` which is stored as a subobject of the parent `operation_state`:
    - Constructs a receiver, `r` and passes `r` to `execution::connect(s, r)` resulting in an operation state `ros`, which is stored as a subobject of `os` such that:
      - When `set_value`, `set_error` or `set_done` is called on `r`, the parameter is copied and stored as a subobject of a receiver `r2` and `execution::connect(execution::schedule(sch), std::move(r2))` results in an `operation_state` `os2` which is stored as a subobject of `os` such that:
        - When `set_value` is called on `r2`, `os2`'s destructor will be called, the stored value is forwarded to `output_receiver` on the appropriate choice of `set_value`, `set_error` or `set_done` to match the operation performed on `r`.
        - When `set_error` or `set_done` is called on `r2` the parameters propagate to `output_receiver`.
    - If `connect` throws, the resulting exception is forwarded to `execution::set_error(output_receiver)`.
    - The destructor of `ros` is called.
    - If `connect` throws, the resulting exception is forwarded to `execution::set_error(output_receiver)`.
    - Calls `execution::start(os2)`.
  - When `execution::start` is called on `os`, call `execution::start(ros)`.
- Otherwise, `execution::on(s, sch)` is ill-formed.
- Any customisation of `execution::on(s, sch)` returning a sender `s2` shall execute calls to `set_value`, `set_done` or `set_error` on a receiver connected to `s2` on an execution context owned by `sch`.

## § 3.6 `execution::when_all`

### § 3.6.1 Overview

`when_all` combines a set of *non-void* senders, returning a sender that, on success, completes with the combined values of all incoming senders. To make usage simpler, `when_all` is restricted to typed\_senders that each send only a single possible value type.

Signature:

```
template <execution::typed_sender Ss...>
see-below when_all(Ss... ss);
```

[ Example:

```
auto r =
    sync_wait(
        transform(
            when_all(just(3) | just(1.2f)),
            [](int a, float b){return a + b;});
    // r==4.2
```

- end example]

### § 3.6.2 Wording

The name `execution::when_all` denotes a customization point object. For some subexpression `ss...`, let `Ss...` be a list of types such that `decltype((ss))...` is `Ss...`. The expression `execution::when_all(ss...)` is expression-equivalent to:

- `when_all(ss...)` if that expression is valid, and if each `Si` in `Ss` satisfies `typed_sender, sender_traits<Si>::value_types<T>` for some type `T` with overload resolution performed in a context that includes the declaration

```
void when_all() = delete;
```

and that does not include a declaration of `execution::when_all`.

- Otherwise, returns a sender, `s`, that, when `connect(s, output_receiver)` is called on the returned sender, for some `output_receiver`, constructs a receiver `ri` for each passed sender, `si` and calls `connect(si, ri)`, returning operation\_state object `osi`. The operation\_states, `osi`, are stored as subobjects within the operation-state object returned from `connect(s, output_receiver)` such that:
  - if `set_value(ti)` is called on all `ri`, for some single value `ti` for each `ri` will concatenate the list of values and call `set_value(output_receiver, t0..., t1..., tn...)`.
  - if `set_done()` is called on any `ri`, will call `set_done(output_receiver)`, discarding other results.
  - if `set_error(e)` is called on any `ri` will call `set_error(output_receiver, e)` for some `e`, discarding other results.

When `start` is called on the returned sender's `operation_state`, call `execution::start(osi)` for each `operation_state` `osi`.

**Note:** See [Planned Developments](#).

## § 3.7 `execution::transform`

### § 3.7.1 Overview

`transform` is a sender adapter that takes a sender and an invocable and returns a sender that propagates the value resulting from calling the invocable on the value passed by the preceding sender.

Signature:

```
template <execution::sender S, moveable-value F>
    requires std::invocable<F, sender_traits<S>::template
value_types>
    see-below transform(S s, F f);
```

[ Example:

```
int r = sync_wait(just(3) | transform([](int v){return v+1;}));
// r==4
```

- end example]

### § 3.7.2 Wording

The name `execution::transform` denotes a customization point object. For some subexpressions `s` and `f`, let `S` be a type such that `decltype((s))` is `S` and `decltype((f))` is `F`. The expression `execution::transform(s, f)` is expression-equivalent to:

- `s.transform(f)` if that expression is valid, `s` satisfies sender.
- Otherwise, `transform(S, F)`, if that expression is valid, `s` satisfies sender with overload resolution performed in a context that includes the declaration

```
void transform() = delete;
```

and that does not include a declaration of `execution::transform`.

- Otherwise constructs a receiver, `r` and passes that receiver to `execution::connect(s, r)` to return an `operation_state` `os` such that:

When some `output_receiver` has been passed to `connect` on the returned sender to return some `operation_state` `os2`:

- If `set_value(r, Ts... ts)` is called, calls `std::invoke(F, ts...)` and passes the result `v` to `execution::set_value(output_receiver, v)`.

- If `f` throws, catches the exception and passes it to `execution::set_error(output_receiver, e)`.
- If `set_error(r, e)` is called, passes `e` to `execution::set_error(output_receiver, e)`.
- If `set_done(r)` is called, calls `execution::set_done(output_receiver)`.

When `start()` is called on `os2` calls `execution::start(os)`.

- Otherwise the expression `execution::transform(s, f)` is ill-formed.

## § 3.8 `execution::let_value`

### § 3.8.1 Overview

`let_value` is a sender adapter that takes a sender and an invocable and returns a sender that keeps the completion result of the incoming sender alive for the duration of the algorithm returned by the invocable and makes that value available to the invocable.

Signature:

```
template<typename F>
struct is-invocable-with {
    template<typename... Args>
    using apply = std::bool_constant<(std::invocable<F, Args...> &&
...)>;
};

template<execution::sender S, moveable-value F>
requires sender_traits<S>::template value_types<
    is-invocable-with<F>::template apply>::value
see-below let_value(S s, F f);
```

where `S<T...>` and `S<T2>` are implementation-defined types that represent senders that send a value of type list `T...` or `T2` respectively in their value channels. Note that in the general case there may be many types `T...` for a given sender, in which case the invocable may have to represent an overload set.

[ Example:

```
int r = sync_wait(
    just(3) |
    let_value([](int& let_v){
        return just(4) | transform([&](int v){return let_v + v;});
    }));
// r==7
```

### § 3.8.2 Wording

The name `execution::let_value` denotes a customization point object. For some subexpressions `s` and `f`, let `S` be a type such that `decltype((s))` is `S` and `decltype((f))` is `F`. The expression `execution::let_value(s, f)` is expression-equivalent to:

- `s.let_value(f)`, if that expression is valid, if `s` satisfies `sender` and `f` satisfies `invocable`.
- Otherwise, `let_value(s, f)`, if that expression is valid, if `s` satisfies `sender` and `f` satisfies `invocable` with overload resolution performed in a context that includes the declaration

```
void let_value() = delete;
```

and that does not include a declaration of `execution::let_value`. \* Otherwise, returns a sender, `s2`, that, when `connect(s, output_receiver)` is called on `s2`, for some `output_receiver`, returning an `operation_state` `os2` which will be stored as a subobject of the parent `operation_state`, constructs a receiver `r` and passes that receiver to `connect(s, r)`, returning `operation_state` object `os` and stores `os` as a subobject of `os2`:

- If `set_value(r, ts...)` is called:
  - copies `ts...` into `os2` as subobjects `t2s...`, calls `std::invoke(f, t2s...)` to return some `invoke_result`
  - calls `execution::connect(invoke_result, output_receiver)` resulting in some `operation_state` `os3`, stores `os3` as a subobject of `os2` and calls `execution::start(os3)`.
  - the destructor of `os2` must be sequenced after the completion of the operation represented by `invoke_result`.
  - If `f` or `connect()` throws, catches the exception and passes it to `set_error(output_receiver, e)`.
- If `set_error(r, e)` is called, passes `e` to `set_error(output_receiver, e)`.
- If `set_done(r)` is called, calls `set_done(output_receiver)`.

When `start` is called on `os2`, call `execution::start(os)`.

- Otherwise the expression `execution::let_value(s, f)` is ill-formed.

## § 3.9 **execution::let\_error**

### § 3.9.1 Overview

`let_error` is a sender adapter that takes a sender and an invocable and returns a sender that, on error propagation, keeps the error result of the incoming sender alive for the duration of the sender returned by the invocable and makes that value available to the invocable.

Signature:

```
template<typename F>
struct is-invocable-with {
    template<typename... Args>
```

```

        using apply = std::bool_constant<(std::invocable<F, Args...> &&
...)>;
    };

    template<execution::sender S, moveable-value F>
    requires sender_traits<S>::template error_types<
        is_invocable_with<F>::template apply>::value
    see-below let_error(S s, F f);

```

[ Example:

```

float r = sync_wait(
    just(3) |
    transform([](int v){throw 2.0f;}) |
    let_error([](float e){return just(e+1);}));
// r==3.0f

```

### § 3.9.2 Wording

The name `execution::let_error` denotes a customization point object. For some subexpressions `s` and `f`, let `S` be a type such that `decltype((s))` is `S` and `decltype((f))` is `F`. The expression `execution::let_error(s, f)` is expression-equivalent to:

- `s.let_error(f)`, if that expression is valid, if `s` satisfies sender.
- Otherwise, `let_error(s, f)`, if that expression is valid, if `s` satisfies sender with overload resolution performed in a context that includes the declaration

```
void let_error() = delete;
```

and that does not include a declaration of `execution::let_error`.

- Otherwise, returns a sender, `s2`, that, when `connect(s, output_receiver)` is called on `s2`, for some `output_receiver`, returning an `operation_state` `os2`, constructs a receiver `r` and passes that receiver to `connect(s, r)`, returning `operation_state` object `os` and stores `os` as a subobject of `os2`:
  - If `set_value(r, ts...)` is called, passes `ts...` to `set_value(output_receiver, ts...)`.
  - If `set_error(r, e)` is called:
    - copies `e` into `os2` as `e2`, calls `std::invoke(f, e2)` to return some `invoke_result`
    - calls `execution::connect(invoke_result, output_receiver)` resulting in some `operation_state` `os3`, stores `os3` as a subobject of `os2` and calls `execution::start(os3)`.
    - the destructor of `os2` must be sequenced after the completion of the operation represented by `invoke_result`.
    - If `f` or `connect()` throws, catches the exception as `e3` and passes it to `set_error(output_receiver, 3)`.



— If `set_done(r)` is called, calls `set_done(output_receiver)`.

When `start` is called on `os2`, call `execution::start(os)`.

— Otherwise the expression `execution::let_error(s, f)` is ill-formed.

## § 3.10 `execution::ensure_started`

### § 3.10.1 Overview

`ensure_started` is a sender adapter that takes a sender, eagerly submits it and returns a sender that propagates the value by reference and can be used as an l-value.

Signature:

```
template <execution::sender S>
see-below ensure_started(S s);
```

[ Example:

```
auto s1 = just(3) | ensure_started();
auto s2 = s1 | transform([](const int& a){return a+1;});
int r = sync_wait(
    transform(
        s2,
        [](int b){return b*2;});
    // r==8
```

### § 3.10.2 Wording

The name `execution::ensure_started` denotes a customization point object. For some subexpressions `s`, let `S` be a type such that `decltype((s))` is `S`. The expression `execution::ensure_started(s, f)` is expression-equivalent to:

- `s.ensure_started()` if that expression is valid and if `s` satisfies sender.
- Otherwise, `ensure_started(s)`, if that expression is valid, if `s` satisfies sender with overload resolution performed in a context that includes the declaration

```
void ensure_started() = delete;
```

and that does not include a declaration of `execution::ensure_started`.

- Otherwise, returns a sender, `s2`, that, constructs a shared state `shr`, constructs a receiver, `r` and passes that receiver to `execution::connect(s, r)` resulting in an operation\_state `os` that is stored as a subobject of `shr`.
  - If `set_value(r, ts)` is called stores `ts` as subobjects of `os`.
  - If `set_error(r, e)` is called, stores `e` as a subobject of `os`.
  - If `set_done(r)` is called stores the done result as a subobject of `os`.

When some `output_receiver` has been passed to connect on `s2`, resulting in an `operation_state os2` and one of the above has been called on `r`:

- If `r` was satisfied with a call to `set_value`, call `set_value(output_receiver, ts...)`
- If `r` was satisfied with a call to `set_error`, call `set_error(output_receiver, e)`.
- If `r` was satisfied with a call to `set_done`, call `execution::set_done(output_receiver)`.
- When `start` is called on `os2`, call `execution::start(os)`.
- If `s2` is destroyed before `start` is called on `os2`, calls `std::terminate()`.

**Note:** See [Planned Developments](#).

## § 4 Customization and example

Each of these algorithms, apart from `just`, is customizable on one or more sender implementations. This allows full optimization. For example, in the following simple work chain:

```
auto s = just(3) |                                     // s1
    on(scheduler1) |                                   // s2
    transform([](int a){return a+1;}) |                 // s3
    transform([](int a){return a*2;}) |                 // s4
    on(scheduler2) |                                   // s5
    let_error([](auto e){return just(3);});             // s6
int r = sync_wait(s);
```

The result of `s1` might be a `just_sender<int>` implemented by the standard library vendor.

`on(just_sender<int>, scheduler1)` has no customization defined, and this expression returns an `scheduler1_on_sender<int>` that is a custom type from the author of `scheduler1`, it will call `submit` on the result of `s1`.

`s3` calls `transform(scheduler1_on_sender<int>, [](int a){return a+1;})` for which the author of `scheduler1` may have written a customization. The `scheduler1_on_sender` has stashed the value somewhere and build some work queue in the background. We do not see `submit` called at this point, it uses a behind-the-scenes implementation to schedule the work on the work queue. An `scheduler1_transform_sender<int>` is returned.

`s4` implements a very similar customization, and again does not call `submit`. There need be no synchronization in this chain.

At s5, however, the implementor of scheduler2 does not know about the implementation of scheduler1. At this point it will call `submit` on the incoming `scheduler1_transform_sender`, forcing scheduler1's sender to implement the necessary synchronization to map back from the behind-the-scenes optimal queue to something interoperable with another vendor's implementation.

`let_error` at s6 will be generic in terms of `submit` and not do anything special, this uses the default implementation in terms of `submit`. `sync_wait` similarly constructs a `condition_variable` and a temporary `int`, submits a receiver to `s` and waits on the `condition_variable`, blocking the calling thread.

`r` is of course the value 8 at this point assuming that neither scheduler triggered an error. If there were to be a scheduling error, then that error would propagate to `let_error` and `r` would subsequently have the value 3.

## § 5 Planned developments

Future changes and discussion points based on R3 of this paper.

### § 5.1 `when_all` and `ensure_started`'s contexts

Based on experience in Facebook's codebase, we believe that `when_all` and `ensure_started` should return senders that require a `scheduler_providers` and use forward progress delegation as discussed in [P1898R1].

In the case of `when_all`, the context the returned sender completes on will depend on which incoming sender completes last. It is thus non-deterministic across that set.

`ensure_started` is similarly adding non-determinism by removing laziness. If the sender returned by `ensure_started` is complete by the time a receiver is connected to it, the `start` call would complete inline with the caller.

In both cases, requiring a `scheduler_provider`, as discussed in [P1898R1] would offer determinism by guaranteeing a transition onto some downstream scheduler and adding wording to require submission onto that provided scheduler if it does not match the completing context.

### § 5.2 `when_all` for void types and mixed success

We should add a `when_all` variant that returns tuples and variants in its result, or some similar mechanism for to allow parameter packs, including empty packs in the form of void-senders, and mixed success/error to propagate.

### § 5.3 `when_all` and `ensure_started` both require cancellation and async cleanup to be fully flexible

Under error circumstances, `when_all` should cancel the other incoming work. This will be described separately.

`ensure_started` similarly needs to be updated to describe how it behaves in the presence of one downstream task being cancelled, and precisely when and where the shared state is destroyed. This would be a preferable solution to termination, as described above, particularly in cases where `ensure_started` is used as part of a set of operations where something else might throw and cause the sender to be destroyed.

## § 6 Proposed question for the Prague 2020 meeting

### § 6.1 Replace `bulk_execute` in P0443 with `indexed_for` as described above.

`indexed_for` as described above should replace `bulk_execute` during the merge of [P0443R11] into C++23. Suggest fine-tuning this wording and forwarding to LEWG.

The changes this leads to:

- Renames the algorithm to fit in with a set of user-level algorithms rather than making it distinct and internal-only. We found it hard to define a difference between `bulk_execute` and `indexed_for` and so suggest we not try, instead we rename it.
- Propagating the data from the incoming sender into the invocable by reference and out the other end. This allows the algorithm to be a side-effecting view on data, but because that data is in-band in the data stream it is safe from a lifetime point of view. More so that it would be if the data had to be captured by reference.
- Replaces the max value with a range for the index space. This allows for more flexibility.
- Adds the execution policy back in, defining the forward progress guarantee both the invocable and range accessor make. This is preferred because the policy is a statement the programmer makes about the capabilities of the invocable. An `indexed_for` that requires `seq`, and an executor that cannot execute `seq` can fail at this point. An invocable that requires `seq` run on an executor that cannot run `seq` algorithms would be invisible at the point of chaining the algorithm.
- Does not add any sort of factory as [P1993R0]. These are not necessary if we carry the data in the stream. Data can be allocated to a device using, for example, a `device_vector`. This maintains full flexibility - we can add custom data management algorithms independently and keep `indexed_for` focused on its primary use cause: the asynchronous for loop itself.
- Relaxes the CopyConstructible restriction that [P0443R11], but also the standard algorithms in C++20 place. Wide discussion suggests that this restriction may not be necessary, and it could certainly be harmful. In an asynchronous world we cannot

rely on scoped `reference_wrapper` semantics, and the cost of injecting `shared_ptr` would be high. If an implementation needs to copy, then that implementation should implement a wrapper that is custom for the algorithmic structure it is using. For example, a forking tree of threads may allocate once on the first thread by move and reference back to it, knowing the lifetime is safe.

## § 6.2 Result of discussion and Prague SG1 vote on P1897R2

Poll: We should add a sender argument and sender result to bulk execution functions (providing an opportunity to build shared state, established dependencies in/out)

SA: 17; F: 7; N: 0; A: 0; SA: 0

Consensus.

Poll: We should replace `bulk_execute` with `indexed_for`

SA: 4; F: 11; N: 3; A: 7; SA: 1

No consensus for change. Discussed in the room that `indexed_for` (and other algorithms by inference) should be build on top of `bulk_execute`.

The `bulk_execute` primitive should take an execution policy to constrain the invocable.

SA: 5; F: 7; N: 8; A: 3; SA: 1

R3 of this paper removes `indexed_for`. If `bulk_execute` is to remain, there is less urgent need to add `indexed_for`. Instead R3 focuses on the core set of algorithms. Something like `indexed_for`, or `for_each` will be in the async update of the parallel algorithms.

## § 7 References

- [P0443R11] 2019. A Unified Executors Proposal for C++.  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0443r11.html>
- [P0443R13] 2020. A Unified Executors Proposal for C++.  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0443r13.html>
- [P1660R0] 2019. A Compromise Executor Design Sketch.  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1660r0.pdf>
- [P1898R1] 2020. Forward progress delegation for executors.  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1898r1.html>
- [P1993R0] 2019. Restore factories to `bulk_execute`.  
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1993r0.pdf>